

# Week 4 Assignment

## Concurrency and Transactions in Backend Systems

In Assignment 3, you implemented a working backend system using FastAPI and SQLAlchemy. Depending on the track you followed, you built either a **Mini Commerce Backend (Ateko Track)** or a **Mini Banking Backend (RBC Track)**.

In those systems, you implemented APIs for retrieving data, pagination, analytics, and multi-step operations such as **order creation** or **money transfers**. These operations involved multiple database actions, such as validating inputs, updating records, and inserting related data across several tables.

For the purpose of that assignment, you were **explicitly asked not to implement transaction management** in those operations. The goal was to focus on building the API logic and understanding how different components of a backend system interact with the database.

However, the implementations from Assignment 3 contain a hidden problem that appears in real-world systems: **concurrency issues**.

In real production environments, many users may interact with the system simultaneously. Multiple API requests can arrive at the same time and attempt to modify the same data. Without proper transaction control and isolation, these concurrent operations may lead to inconsistent or incorrect results.

Examples include:

- Two customers purchasing the same product at the same time when only one item is left in stock.
- Two banking transfers reading the same account balance simultaneously and producing an incorrect final balance.
- Multiple requests modifying related data where one step fails but earlier steps were already written to the database.

To prevent these problems, backend systems rely on database transactions and isolation levels. Transactions ensure that a sequence of database operations behaves as a single logical unit of work, meaning that either all changes succeed together or none of them are applied. Isolation levels control how concurrent transactions interact with each other, preventing issues such as dirty reads, lost updates, and inconsistent data visibility.

**This assignment builds directly on your previous implementation. You will analyze the behavior of your existing system under concurrent access, identify potential problems, and update your implementation to make it safe and reliable using transactions.**

Your work will focus on three key goals:

Understanding how race conditions occur in backend APIs

Applying transaction management to multi-step operations

Reasoning about isolation levels and concurrency behavior in real systems

You are encouraged to reuse and extend the code you wrote in Assignment 3. The purpose of this assignment is not to rebuild the system, but to improve its correctness and reliability under concurrent workloads.

## How to Start?

To complete this assignment, you will continue working with the **same track you selected in Assignment 3**.

In the previous assignment, you chose one of the following tracks:

- **Track A – Ateko Project: Mini Commerce Backend**
- **Track B – RBC Project: Mini Banking Backend**

For this assignment, you should return to the same project and reuse the code you already developed.

Inside the **Week 4 folder on Brightspace**, you will find two instruction files:

- **Track\_A\_Week4\_Instructions**
- **Track\_B\_Week4\_Instructions**

Open the file that corresponds to the **track you worked on in Assignment 3** and follow the instructions provided there. Each track contains scenarios and tasks specifically designed for the system you previously implemented.

You are encouraged to reuse your existing codebase and make the required modifications directly in that project. The goal of this assignment is to improve the reliability of your existing backend by applying concepts such as transactions, concurrency control, and isolation behavior.

## Expected Time

This assignment is designed to be a **short applied exercise rather than a large implementation task**. Most of the work involves reasoning about concurrency and making small adjustments to your existing code.

The assignment should take **approximately 30 to 60 minutes** to complete, depending on how comfortable you are with the concepts discussed in the Week 4 session.

## Grading Rubric (Shared for Both Tracks)

This assignment evaluates your understanding of **concurrency issues, transaction management, and isolation behavior in backend systems**, as discussed in the Week 4 session.

Total: **100%**

| Section                             | Criteria                                                                                                                                                                                          | Weight     |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Part 1 – Concurrency Analysis       | Correctly identifies the incorrect outcome caused by concurrent requests and explains the race condition or lost update scenario.                                                                 | <b>20%</b> |
| Part 2 – Transaction Implementation | Proper implementation of transaction management in the critical API endpoint ( <code>POST /orders</code> or <code>POST /transfers</code> ). Ensures that multi-step operations behave atomically. | <b>35%</b> |
|                                     | Correct handling of commit and rollback behavior to prevent partial database updates.                                                                                                             | <b>10%</b> |
| Part 3 – Isolation Level Reasoning  | Demonstrates understanding of isolation levels and explains trade-offs between consistency and performance.                                                                                       | <b>15%</b> |
| Part 4 – Concurrency Reflection     | Explains how transactions and connection pooling help maintain system reliability under concurrent workloads.                                                                                     | <b>10%</b> |
| Code Quality and Functionality      | Application runs successfully and transaction logic integrates correctly with the existing backend project.                                                                                       | <b>10%</b> |